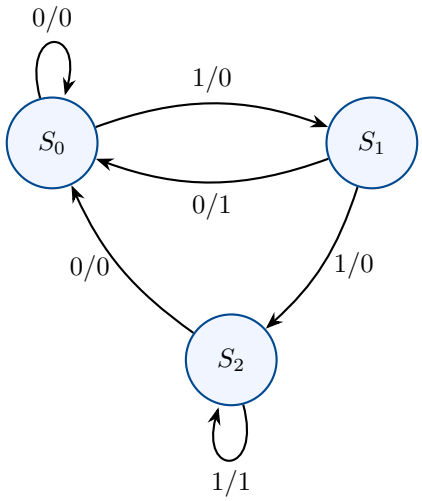
Finite State Machine Design Guide

From Fundamentals to Advanced Real-World Digital Systems



Digital System Design — Advanced Level

March 4, 2026

Contents

1 Introduction to Finite State Machines **3**
1.1 Moore vs. Mealy Machines . . . 3
1.2 General FSM Architecture . . . 3

2 FSM Design Methodology — Step by Step **4**
2.1 D Flip-Flop Characteristic and Excitation . . . 4
2.2 State Encoding Techniques . . . 5

3 Common Errors in FSM Design **5**

4 Best Practices for FSM Design **6**

5 From State Diagram to D Flip-Flop Implementation **7**
5.1 Example: Sequence Detector (detects "101") . . . 7
5.1.1 Step 1: State Diagram . . . 7
5.1.2 Step 2: State Assignment . . . 7
5.1.3 Step 3: State / Transition Table . . . 8
5.1.4 Step 4: Excitation Table (D Flip-Flops) . . . 8
5.1.5 Step 5: K-Map Simplification . . . 8
5.1.6 Step 6: Circuit Diagram . . . 9

6 FSM Design Problems: Beginner to Advanced **9**
6.1 Problem 1: Simple Toggle Switch (Beginner) . . . 9
6.2 Problem 2: 2-Bit Up Counter (Beginner) . . . 10
6.3 Problem 3: Sequence Detector "11" (Beginner) . . . 11
6.4 Problem 4: 3-Bit Up/Down Counter (Beginner–Intermediate) . . . 11
6.5 Problem 5: Vending Machine Controller (Intermediate) . . . 12
6.6 Problem 6: Traffic Light Controller (Intermediate) . . . 13
6.7 Problem 7: Serial Adder (Intermediate) . . . 14
6.8 Problem 8: Pattern Detector "1010" Non-Overlapping (Intermediate) . . . 15
6.9 Problem 9: Elevator Door Controller (Intermediate–Advanced) . . . 16
6.10 Problem 10: SPI Master Controller (Advanced) . . . 17
6.11 Problem 11: UART Receiver (Advanced) . . . 18
6.12 Problem 12: Washing Machine Controller (Advanced) . . . 19
6.13 Problem 13: Keypad Debouncer / Lock (Advanced) . . . 20
6.14 Problem 14: Memory Read Controller (Advanced) . . . 21
6.15 Problem 15: AHB-Lite Bus Master (Expert) . . . 21

7 Complete Worked Example: Excitation Table to Circuit **22**
7.1 Given State Diagram . . . 22
7.2 Complete Excitation Table . . . 23
7.3 K-Map for D_1 . . . 23
7.4 K-Map for D_0 . . . 23
7.5 Output K-Maps . . . 23
7.6 Final Circuit . . . 24

8 Comparison: Moore vs. Mealy Implementation **24**

9 Advanced Topics **24**
9.1 One-Hot Encoding Implementation . . . 24
9.2 State Minimization Using Implication Chart . . . 25
9.3 Handling Unused States . . . 25
9.4 Problem 16: Sequential Button Door Lock (Expert — Full Design) . . . 25
9.4.1 Problem Statement . . . 25
9.4.2 Understanding Output A (Activity Indicator) . . . 26
9.4.3 State Definitions . . . 27
9.4.4 State Diagram . . . 27
9.4.5 Edge Cases — Every Scenario Explained . . . 28
9.4.6 State Assignment (Binary Encoding) . . . 29
9.4.7 Complete Excitation Table . . . 30
9.4.8 Output Equations . . . 31
9.4.9 Timing Diagram — Correct Sequence . . . 31
9.4.10 Timing Diagram — Wrong Button Scenario . . . 32
9.4.11 Strict vs. Lenient Design Comparison . . . 32

10 VHDL Template for FSM Design **32**
10.1 The Three-Process FSM Template . . . 33
10.2 Generic VHDL FSM Template . . . 33
10.3 Template Rules — Never Break These . . . 35
10.4 Applied Template: Button Lock in VHDL . . . 36

11 Quick Reference: FSM Design Checklist **38**

12 Summary of All 16 Problems **39**

1 Introduction to Finite State Machines

A **Finite State Machine (FSM)** is a mathematical model of computation consisting of:

- A finite set of **states** $S = \{S_0, S_1, \ldots, S_{n-1}\}$
- A finite **input alphabet** Σ
- A **transition function** $\delta : S \times \Sigma \to S$
- An **output function** (Moore or Mealy)
- An **initial state** S_0

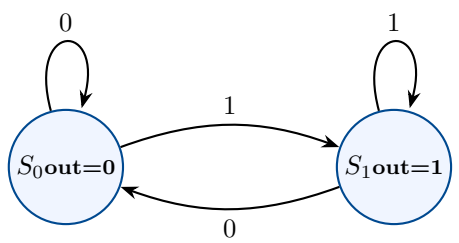
1.1 Moore vs. Mealy Machines

Moore Machine

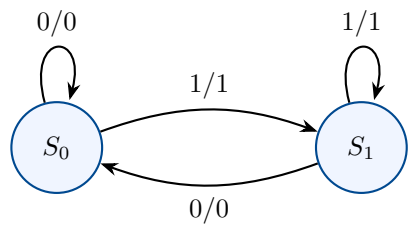
Output depends **only** on the current state.
$\lambda : S \to \Gamma$ (output function)
Outputs change synchronously with state transitions.

Mealy Machine

Output depends on **both** the current state **and** input.
$\lambda : S \times \Sigma \to \Gamma$ (output function)
Outputs can change asynchronously with input changes.



Moore Machine: output in state



Mealy Machine: input/output on edges

1.2 General FSM Architecture

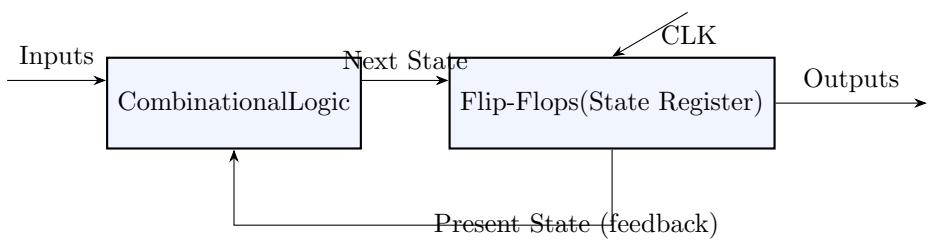


Figure 1: General FSM Block Diagram

2 FSM Design Methodology — Step by Step

The Golden 8-Step Process

1. **Understand the problem** — identify inputs, outputs, and behaviour.
2. **Draw the state diagram** — all states, transitions, outputs.
3. **Minimize states** — merge equivalent states (implication chart).
4. **Assign state encoding** — binary, Gray, one-hot.
5. **Build the state/transition table** — present state, input, next state, output.
6. **Derive the excitation table** — using D flip-flop characteristics.
7. **Obtain Boolean equations** — K-maps for each flip-flop input and output.
8. **Draw the circuit** — connect flip-flops, gates, and I/O.

2.1 D Flip-Flop Characteristic and Excitation

The D flip-flop is the simplest memory element for FSM implementation.

Table 1: D Flip-Flop Characteristic Table

D	CLK↑	Q^+ (Next State)
0	↑	0
1	↑	1

Table 2: D Flip-Flop Excitation Table

Q (Present)	Q^+ (Next)	D (Required Input)
0	0	0
0	1	1
1	0	0
1	1	1

Key Insight

For a D flip-flop: $D = Q^+$. The required D input is simply equal to the desired next state. This makes D flip-flops the easiest to use in FSM design!

2.2 State Encoding Techniques

Table 3: Comparison of State Encoding Methods

Encoding	Flip-Flops	Pros	Cons
Binary	$\lceil \log_2 n \rceil$	Fewest flip-flops	Complex next-state logic
Gray Code	$\lceil \log_2 n \rceil$	1-bit change per transition	Limited applicability
One-Hot	n	Simplest logic, fast	Many flip-flops

3 Common Errors in FSM Design

Critical Mistakes to Avoid

1. **Missing transitions:** Every state must have a defined transition for *every* possible input combination. Missing transitions lead to undefined/stuck states.
2. **Missing reset state:** Every FSM *must* have a defined initial/reset state. Without it, the FSM powers up in an unknown state.
3. **Unreachable states:** States that cannot be reached from the initial state waste resources and may cause problems if entered accidentally.
4. **Dead-end states (latch-up):** States with no outgoing transitions trap the FSM forever.
5. **Wrong state encoding:** Incorrect binary assignment leads to wrong excitation equations and faulty operation.
6. **Glitches in Mealy outputs:** Since Mealy outputs depend on inputs, asynchronous input changes can cause output glitches.
7. **Incorrect excitation table:** Confusing characteristic table with excitation table, or using JK/T excitation for D flip-flops.
8. **Don't-care mishandling:** Unused states should be assigned don't-care or safe transitions, not left floating.
9. **Clock domain issues:** Inputs not synchronized to the clock domain cause metastability.
10. **State minimization errors:** Merging non-equivalent states changes the FSM's behaviour.
11. **Output equation errors:** Computing output from next state instead of present state (Moore), or forgetting input dependency (Mealy).
12. **Forgetting to handle unused states in implementation:** In a 3-flip-flop design with 5 used states, the 3 unused states (binary combinations) must transition to a safe state.

4 Best Practices for FSM Design

Professional Design Guidelines

1. **Always start with a clear specification** — list all inputs, outputs, and describe every scenario before drawing anything.
2. **Draw the state diagram first** — visual representation catches errors early.
3. **Label everything** — states, transitions (input/output), and reset arrows.
4. **Check completeness** — verify every state has transitions for all input combinations.
5. **Handle unused states explicitly** — assign them transitions to the reset state.

6. **Use D flip-flops unless there's a specific reason not to** — $D = Q^+$ simplifies everything.
7. **Verify with a timing diagram** — trace signals through time to validate.
8. **Use simulation before hardware** — ModelSim, Vivado, etc.
9. **Prefer Moore when possible** — glitch-free synchronous outputs.
10. **Document your encoding** — always write down which binary code maps to which state.

5 From State Diagram to D Flip-Flop Implementation

This section walks through the **complete methodology** using a detailed example.

5.1 Example: Sequence Detector (detects "101")

Specification: Design a Moore FSM that outputs 1 when the sequence "101" is detected on a serial input X. Overlapping sequences allowed.

5.1.1 Step 1: State Diagram

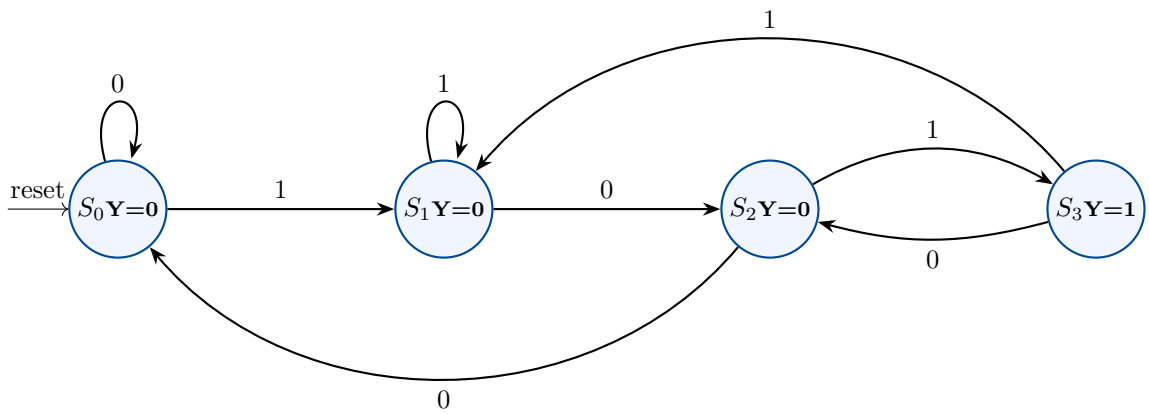


Figure 2: State Diagram for "101" Sequence Detector (Moore)

5.1.2 Step 2: State Assignment

With 4 states we need $\lceil \log_2 4 \rceil = 2$ flip-flops (Q_1, Q_0).

State	Q_1	Q_0
S_0	0	0
S_1	0	1
S_2	1	0
S_3	1	1

Table 4: State Encoding

5.1.3 Step 3: State / Transition Table

Table 5: State Transition Table

Present State		Next State		Output
Q_1Q_0	X	$Q_1^+Q_0^+$	State	Y
00 (S_0)	0	00	S_0	0
00 (S_0)	1	01	S_1	0
01 (S_1)	0	10	S_2	0
01 (S_1)	1	01	S_1	0
10 (S_2)	0	00	S_0	0
10 (S_2)	1	11	S_3	0
11 (S_3)	0	10	S_2	1
11 (S_3)	1	01	S_1	1

5.1.4 Step 4: Excitation Table (D Flip-Flops)

Since $D = Q^+$, the excitation table is straightforward:

Table 6: Excitation Table for D Flip-Flops

Q_1	Q_0	X	Q_1^+	Q_0^+	D_1	D_0
0	0	0	0	0	0	0
0	0	1	0	1	0	1
0	1	0	1	0	1	0
0	1	1	0	1	0	1
1	0	0	0	0	0	0
1	0	1	1	1	1	1
1	1	0	1	0	1	0
1	1	1	0	1	0	1

5.1.5 Step 5: K-Map Simplification

For D_1:

$Q_1Q_0 \backslash X$	0	1	1	0
00				
01				

K-map for D_1 with variables Q_1, Q_0, X:

$Q_1Q_0 \backslash X$	0	1
00	0	0
01	1	0
11	1	0
10	0	1

K-map for D_1

$Q_1Q_0 \backslash X$	0	1
00	0	1
01	0	1
11	0	1
10	0	1

K-map for D_0

From the K-maps:

$$D_1 = Q_0 \cdot \overline{X} + Q_1 \cdot X \cdot \overline{Q_0} = Q_0\,\overline{X} + Q_1\,\overline{Q_0}\,X \tag{1}$$
$$D_0 = X \tag{2}$$
$$Y = Q_1 \cdot Q_0 \tag{3}$$

5.1.6 Step 6: Circuit Diagram

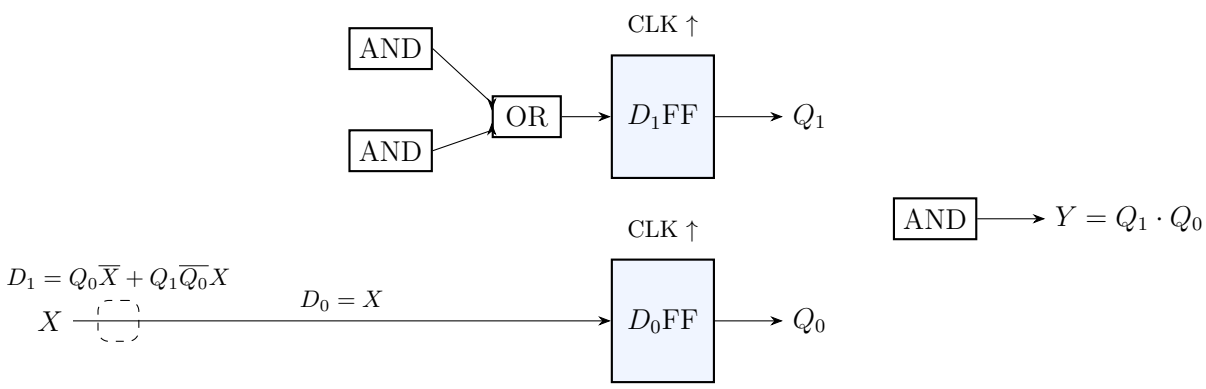


Figure 3: Circuit Implementation for "101" Detector

6 FSM Design Problems: Beginner to Advanced

6.1 Problem 1: Simple Toggle Switch (Beginner)

Problem Statement

Design a Moore FSM with one input T and one output Y. The output toggles (changes state) every time $T = 1$ on a clock edge. When $T = 0$, output remains unchanged. Initial output = 0.

State Diagram:

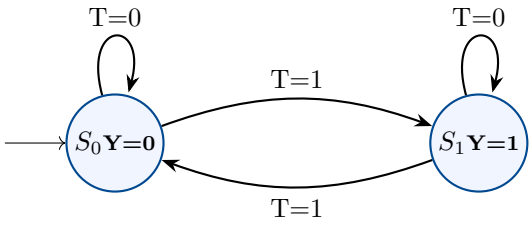


Figure 4: Toggle Switch FSM

State Assignment: $S_0 = 0$, $S_1 = 1$ (1 flip-flop Q)
Transition & Excitation Table:

Q	T	Q^+	D	Y
0	0	0	0	0
0	1	1	1	0
1	0	1	1	1
1	1	0	0	1

Equations:

$$D = Q \oplus T = Q\overline{T} + \overline{Q}T, \quad Y = Q$$

6.2 Problem 2: 2-Bit Up Counter (Beginner)

Problem Statement

Design a 2-bit binary up counter that counts 00 → 01 → 10 → 11 → 00 on each clock edge. Output is the count value.

State Diagram:

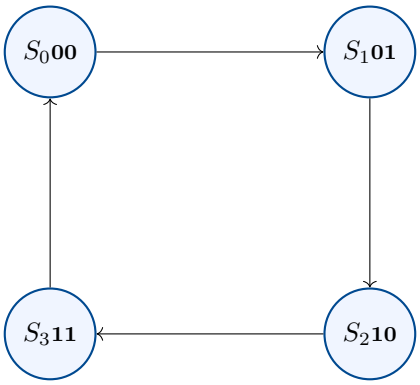


Figure 5: 2-Bit Up Counter

Excitation Table (no external input):

Q_1	Q_0	Q_1^+	Q_0^+	D_1	D_0
0	0	0	1	0	1
0	1	1	0	1	0
1	0	1	1	1	1
1	1	0	0	0	0

Equations:

$$D_1 = Q_1 \oplus Q_0, \quad D_0 = \overline{Q_0}$$

6.3 Problem 3: Sequence Detector "11" (Beginner)

Problem Statement

Design a Mealy FSM that outputs 1 when two consecutive 1's are detected on input X. Overlapping allowed.

State Diagram:

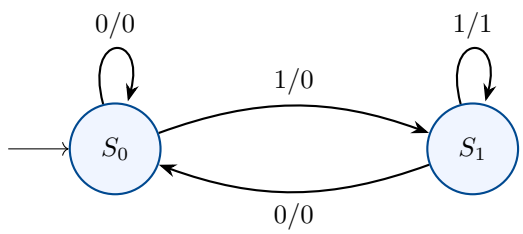


Figure 6: "11" Sequence Detector (Mealy)

State Assignment: $S_0 = 0$, $S_1 = 1$
Excitation Table:

Q	X	Q^+	D	Y
0	0	0	0	0
0	1	1	1	0
1	0	0	0	0
1	1	1	1	1

Equations:

$$D = X, \quad Y = Q \cdot X$$

6.4 Problem 4: 3-Bit Up/Down Counter (Beginner–Intermediate)

Problem Statement

Design a 3-bit counter with input M: counts up when $M = 1$, counts down when $M = 0$.

State Diagram:

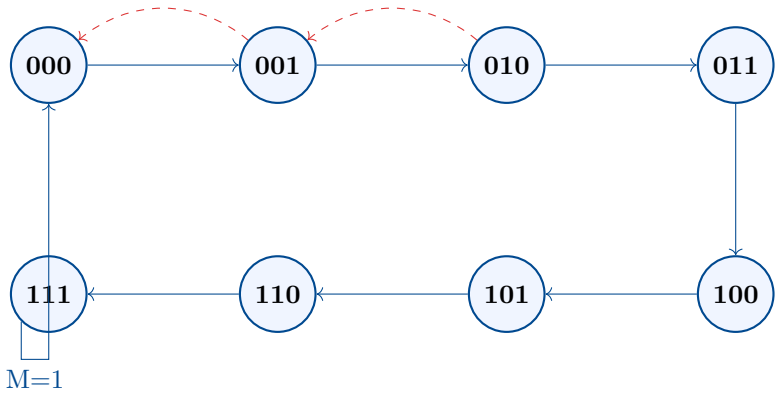


Figure 7: 3-Bit Up/Down Counter (blue=up M=1, red dashed=down M=0)

Equations (derived from standard counter logic):

$$D_2 = Q_2 \oplus (M \cdot Q_1 \cdot Q_0 + \overline{M} \cdot \overline{Q_1} \cdot \overline{Q_0}) \tag{4}$$

$$D_1 = Q_1 \oplus (M \cdot Q_0 + \overline{M} \cdot \overline{Q_0}) \tag{5}$$

$$D_0 = \overline{Q_0} \tag{6}$$

6.5 Problem 5: Vending Machine Controller (Intermediate)

Problem Statement

Design a Moore FSM for a vending machine that accepts 5¢ and 10¢ coins. Item costs 15¢. Output `DISPENSE`=1 when 15¢ or more is collected, then reset. No change given.

States: S_0: 0¢, S_1: 5¢, S_2: 10¢, S_3: 15¢+ (dispense)
Inputs: N (nickel=5¢), D (dime=10¢). Assume mutually exclusive.

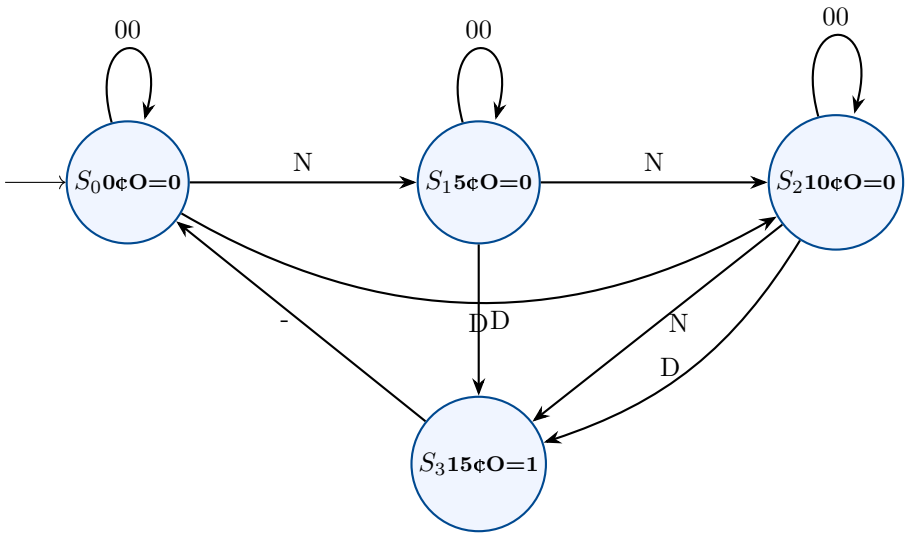


Figure 8: Vending Machine FSM

State Assignment: $S_0 = 00$, $S_1 = 01$, $S_2 = 10$, $S_3 = 11$

Transition Table:

Q_1	Q_0	N	D	Q_1^+	Q_0^+	Dispense
0	0	0	0	0	0	0
0	0	1	0	0	1	0
0	0	0	1	1	0	0
0	1	0	0	0	1	0
0	1	1	0	1	0	0
0	1	0	1	1	1	0
1	0	0	0	1	0	0
1	0	1	0	1	1	0
1	0	0	1	1	1	0
1	1	X	X	0	0	1

Equations (simplified):

$$D_1 = Q_1\,\overline{Q_0} + \overline{Q_1}\,Q_0\,N + \overline{Q_1}\,D + Q_1\,Q_0' \cdot (N + D) \tag{7}$$

$$D_0 = \overline{Q_1}\,\overline{Q_0}\,N + \overline{Q_1}\,Q_0\,\overline{N}\,\overline{D} + Q_1\,\overline{Q_0}\,(N + D) + \overline{Q_1}\,Q_0\,D \tag{8}$$

$$\text{Dispense} = Q_1 \cdot Q_0 \tag{9}$$

6.6 Problem 6: Traffic Light Controller (Intermediate)

Problem Statement

Design a Moore FSM for a simple traffic light: Green (G) → Yellow (Y) → Red (R) → Green. Each state lasts one clock period. A sensor input S keeps green as long as $S = 1$ (cars present).

States: S_G (Green), S_Y (Yellow), S_R (Red)

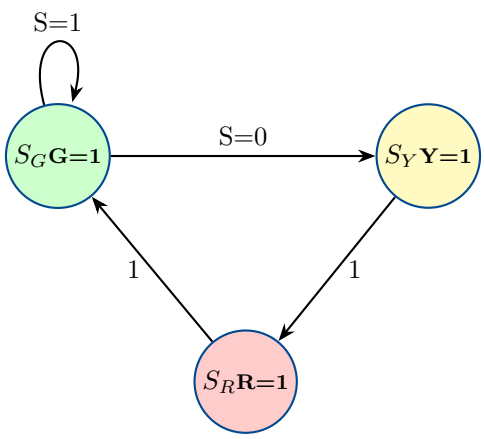


Figure 9: Traffic Light Controller

State Assignment: $S_G = 00$, $S_Y = 01$, $S_R = 10$
Excitation Table:

Q_1	Q_0	S	Q_1^+	Q_0^+	D_1	D_0	G	Y	R
0	0	0	0	1	0	1	1	0	0
0	0	1	0	0	0	0	1	0	0
0	1	X	1	0	1	0	0	1	0
1	0	X	0	0	0	0	0	0	1
1	1	X	0	0	0	0	–	–	–

Equations:

$$D_1 = Q_0 \tag{10}$$

$$D_0 = \overline{Q_1} \cdot \overline{Q_0} \cdot \overline{S} \tag{11}$$

$$G = \overline{Q_1} \cdot \overline{Q_0}, \quad Y = \overline{Q_1} \cdot Q_0, \quad R = Q_1 \cdot \overline{Q_0} \tag{12}$$

6.7 Problem 7: Serial Adder (Intermediate)

Problem Statement

Design a Mealy FSM serial adder. Two serial inputs A and B (LSB first) are added bit by bit. The FSM stores carry as state. Output S = sum bit.

States: S_0 (carry = 0), S_1 (carry = 1)

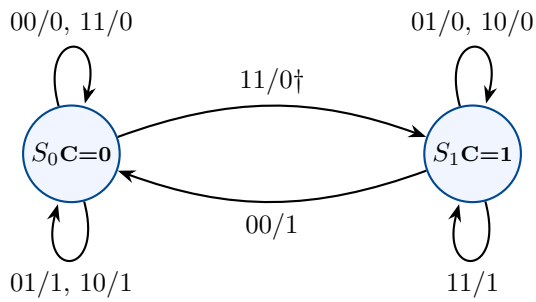


Figure 10: Serial Adder FSM — † transitions corrected below

Full truth table for serial addition: $A + B + C_{in} = \{C_{out}, S\}$

Q (C_{in})	A	B	Sum	C_{out}	D	S
0	0	0	0	0	0	0
0	0	1	1	0	0	1
0	1	0	1	0	0	1
0	1	1	0	1	1	0
1	0	0	1	0	0	1
1	0	1	0	1	1	0
1	1	0	0	1	1	0
1	1	1	1	1	1	1

Equations:

$$D = A \cdot B + A \cdot Q + B \cdot Q = \text{Majority}(A, B, Q) \tag{13}$$
$$S = A \oplus B \oplus Q \tag{14}$$

6.8 Problem 8: Pattern Detector "1010" Non-Overlapping (Intermediate)

Problem Statement

Design a Moore FSM that detects the non-overlapping sequence "1010" on input X. Output $Y = 1$ for one clock cycle upon detection.

States: S_0 (idle), S_1 (got 1), S_2 (got 10), S_3 (got 101), S_4 (got 1010, output)

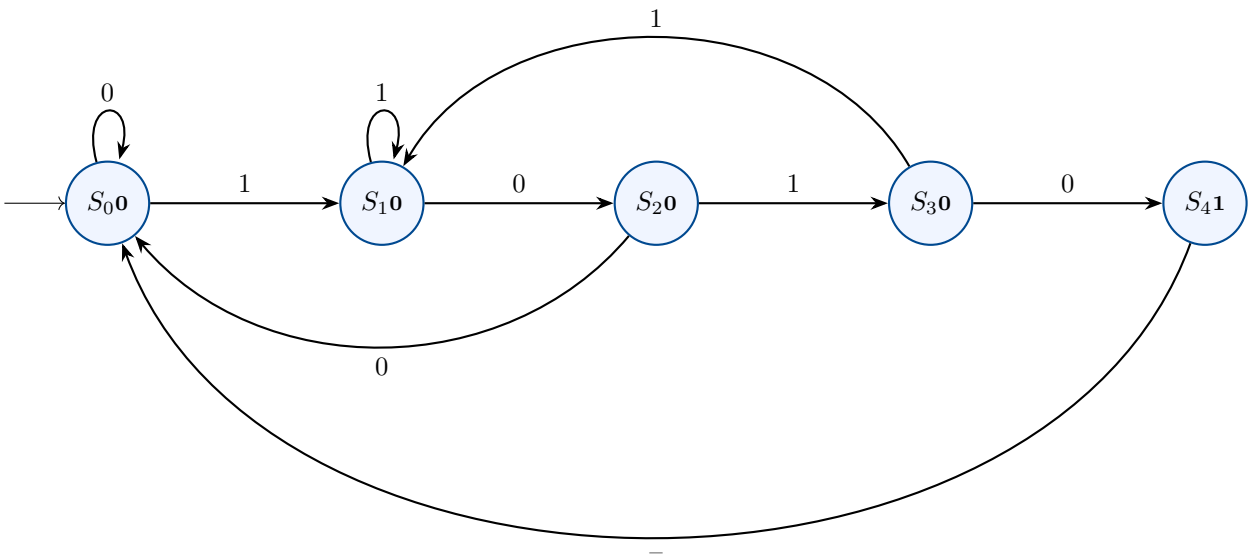


Figure 11: "1010" Non-Overlapping Detector (Moore)

State Assignment (5 states ⇒ 3 flip-flops):

State	Q_2	Q_1	Q_0
S_0	0	0	0
S_1	0	0	1
S_2	0	1	0
S_3	0	1	1
S_4	1	0	0

Excitation Table:

Q_2	Q_1	Q_0	X	Q_2^+	Q_1^+	Q_0^+	D_2	D_1	D_0	Y
0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	1	0	0	1	0
0	0	1	0	0	1	0	0	1	0	0
0	0	1	1	0	0	1	0	0	1	0
0	1	0	0	0	0	0	0	0	0	0
0	1	0	1	0	1	1	0	1	1	0
0	1	1	0	1	0	0	1	0	0	0
0	1	1	1	0	0	1	0	0	1	0
1	0	0	X	0	0	0	0	0	0	1

Unused states (101, 110, 111) $\rightarrow D_2D_1D_0 = 000$ (safe reset).
Equations (from K-maps):

$$D_2 = Q_1 \cdot Q_0 \cdot \overline{X} \tag{15}$$
$$D_1 = Q_0 \cdot \overline{X} \cdot \overline{Q_1} + Q_1 \cdot \overline{Q_0} \cdot X \tag{16}$$
$$D_0 = X \cdot \overline{Q_2} \cdot (\overline{Q_1} + \overline{Q_0} \cdot Q_1) = X \cdot \overline{Q_2} \cdot \overline{Q_1 \cdot Q_0} \tag{17}$$
$$Y = Q_2 \tag{18}$$

6.9 Problem 9: Elevator Door Controller (Intermediate–Advanced)

Problem Statement

Design a Moore FSM for an elevator door with inputs: $OPEN_BTN$, $CLOSE_BTN$, $SENSOR$ (obstacle detected). States: CLOSED, OPENING, OPEN, CLOSING.
Rules: (1) If obstacle detected while closing, go to OPENING. (2) Door stays open while OPEN_BTN pressed. (3) After open completes, wait then close.

Figure 12: Elevator Door Controller FSM

State Assignment: CLOSED=00, OPENING=01, OPEN=10, CLOSING=11.

The excitation and output equations follow from the standard derivation approach. The key insight here is the safety-critical transition: **CLOSING** → **OPENING** when sensor detects an obstacle takes priority over all other transitions.

6.10 Problem 10: SPI Master Controller (Advanced)

Problem Statement

Design a Moore FSM for a simplified SPI master that: (1) waits for a START signal, (2) asserts CS low, (3) shifts out 8 bits (MSB first) on MOSI with SCLK toggling, (4) de-asserts CS and returns to idle.

States:

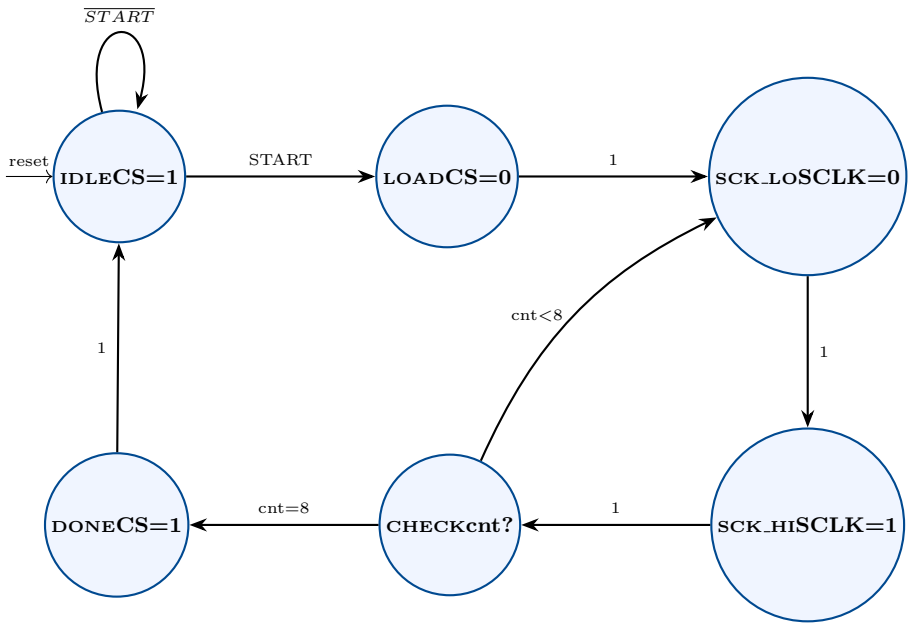


Figure 13: SPI Master Controller FSM

This requires 6 states = 3 flip-flops + a 3-bit counter for bit count. The FSM controls the protocol while the counter tracks progress.

6.11 Problem 11: UART Receiver (Advanced)

Problem Statement

Design a Moore FSM for a UART receiver (8N1 format): detect start bit, sample 8 data bits at mid-bit, check stop bit, assert data-ready.

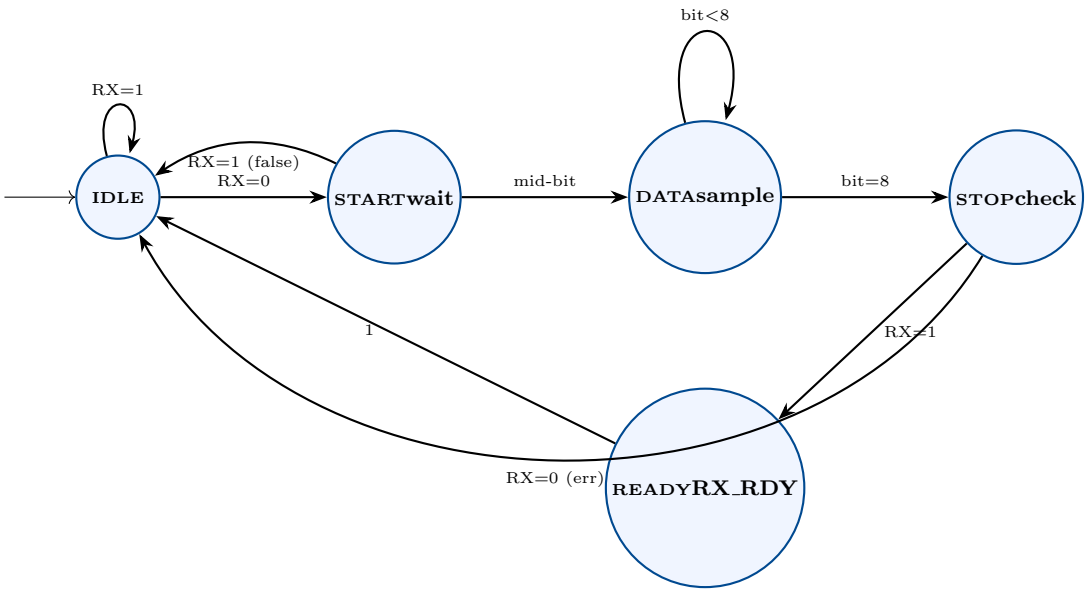


Figure 14: UART Receiver FSM

Implementation notes:

- Requires a baud-rate counter (e.g., 16x oversampling) to find mid-bit timing
- 3-bit counter for tracking bit index (0–7)
- 8-bit shift register for received data
- 5 states → 3 flip-flops for state encoding

6.12 Problem 12: Washing Machine Controller (Advanced)

Problem Statement

Design a Moore FSM for a washing machine with states: IDLE, FILL, WASH, RINSE, SPIN, DONE. Inputs: START, WATER_LEVEL, TIMER_DONE. Outputs: VALVE, MOTOR, PUMP, BUZZER.

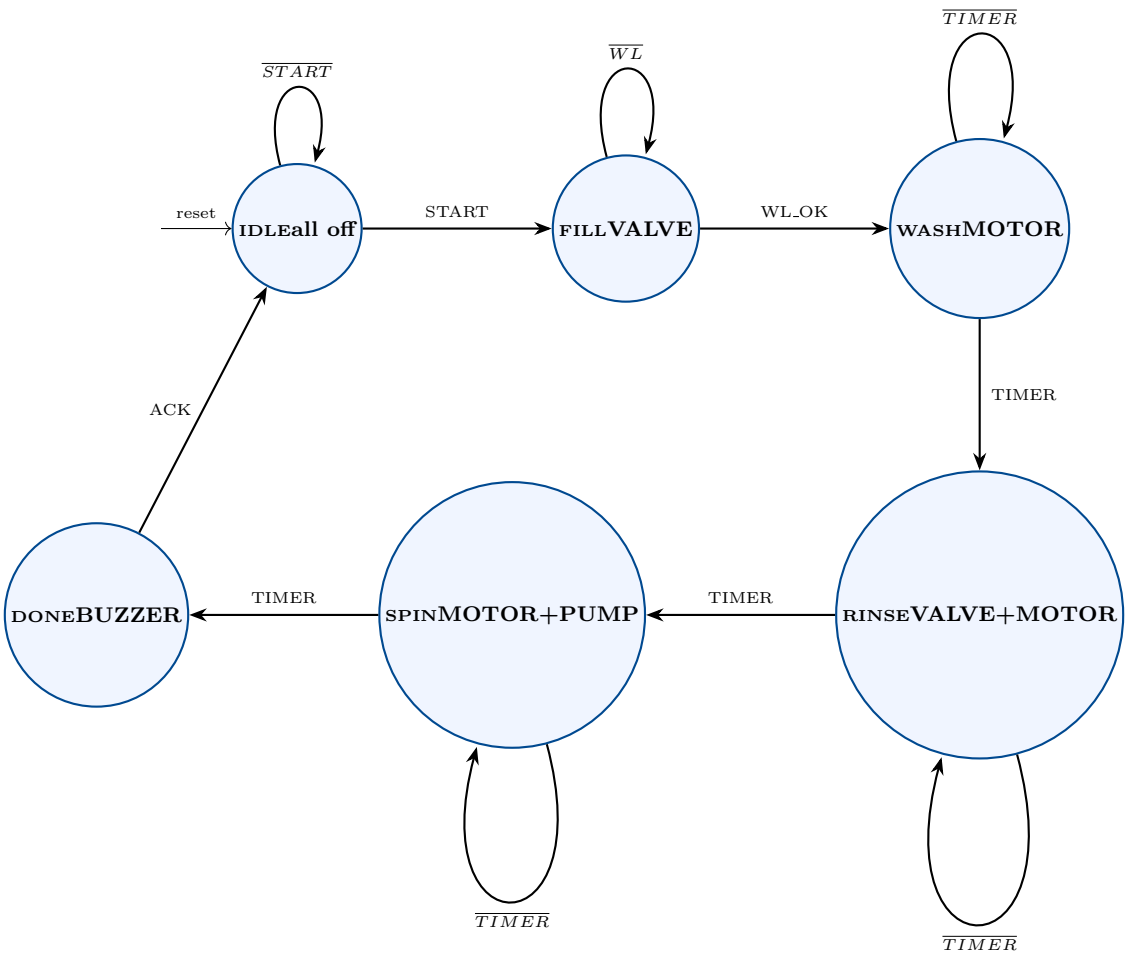


Figure 15: Washing Machine Controller FSM

State Assignment (6 states ⇒ 3 flip-flops):

State	Q_2	Q_1	Q_0	VALVE	MOTOR	PUMP	BUZZER
IDLE	0	0	0	0	0	0	0
FILL	0	0	1	1	0	0	0
WASH	0	1	0	0	1	0	0
RINSE	0	1	1	1	1	0	0
SPIN	1	0	0	0	1	1	0
DONE	1	0	1	0	0	0	1

Output Equations:

$$\text{VALVE} = Q_0 \cdot \overline{Q_2} \tag{19}$$
$$\text{MOTOR} = Q_1 + Q_2 \cdot \overline{Q_0} \tag{20}$$
$$\text{PUMP} = Q_2 \cdot \overline{Q_1} \cdot \overline{Q_0} \tag{21}$$
$$\text{BUZZER} = Q_2 \cdot Q_0 \tag{22}$$

6.13 Problem 13: Keypad Debouncer / Lock (Advanced)

Problem Statement

Design a Mealy FSM for a 2-digit combination lock. The correct code is "AB" (two sequential inputs). If wrong digit entered at any stage, return to start. Output UNLOCK=1 only after correct sequence.

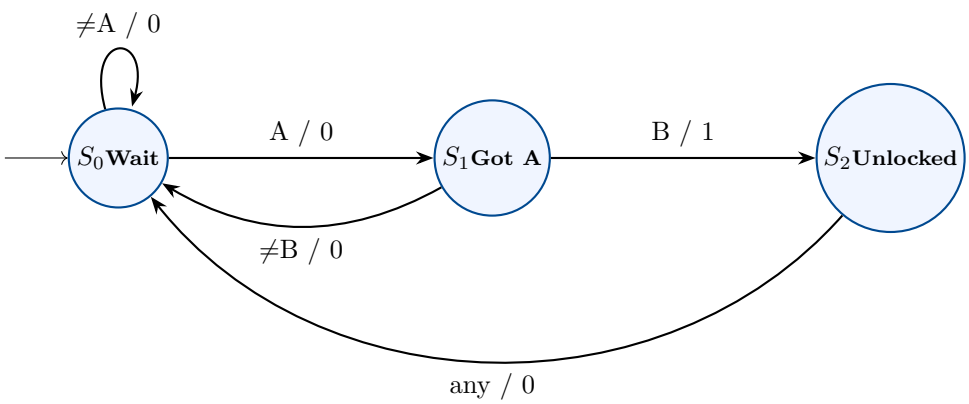


Figure 16: Combination Lock FSM

With 3 states, we need 2 flip-flops. The input needs an encoder to convert keypad data to comparison signals.

6.14 Problem 14: Memory Read Controller (Advanced)

Problem Statement

Design a Moore FSM for a memory read controller that interfaces with SRAM. Sequence: IDLE → assert address and RD → wait for ACK → latch data → IDLE.

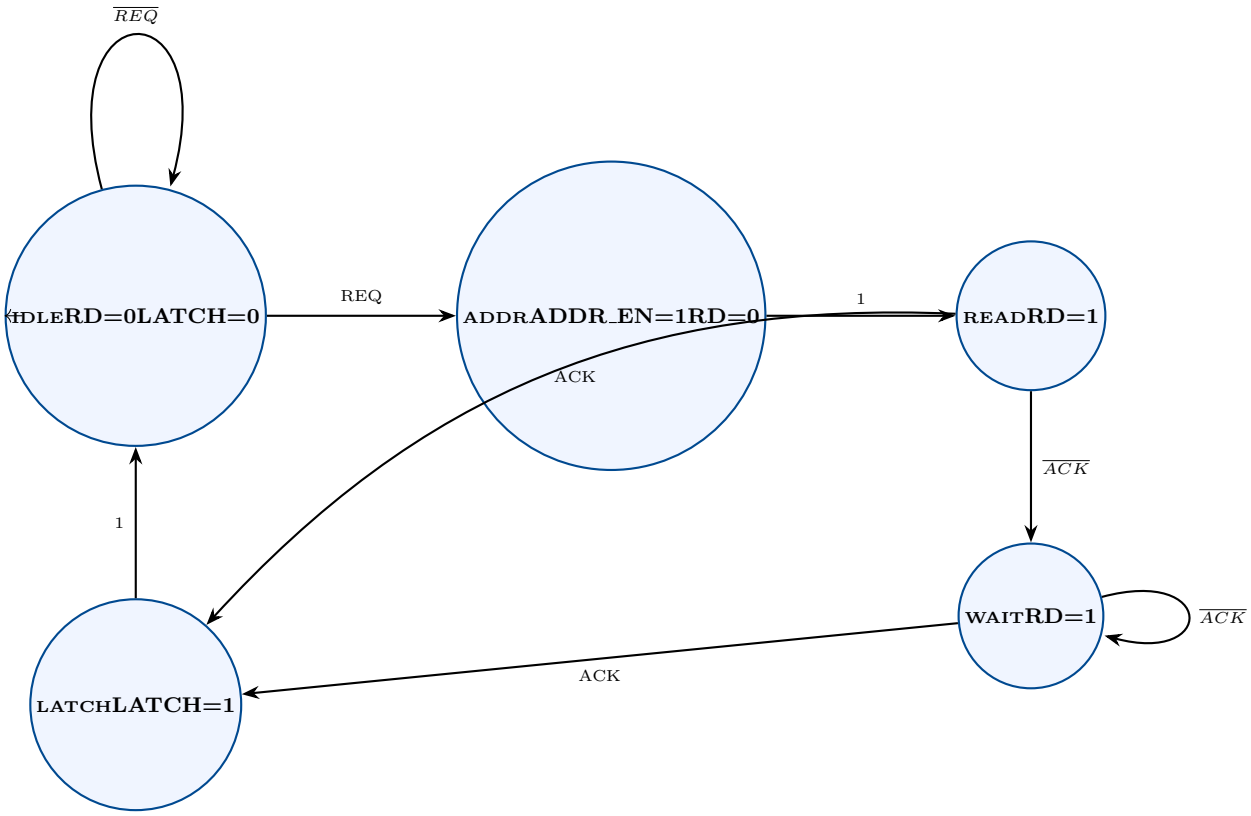


Figure 17: Memory Read Controller FSM

State Assignment (5 states ⇒ 3 flip-flops): IDLE=000, ADDR=001, READ=010, WAIT=011, LATCH=100.

6.15 Problem 15: AHB-Lite Bus Master (Expert)

Problem Statement

Design a Moore FSM for a simplified AHB-Lite bus master with pipelined transfers.
States: IDLE, ADDR_PHASE, DATA_PHASE, WAIT_HREADY, ERROR.
Inputs: HTRANS_REQ, HREADY, HRESP.
Outputs: HTRANS[1:0], control signals.

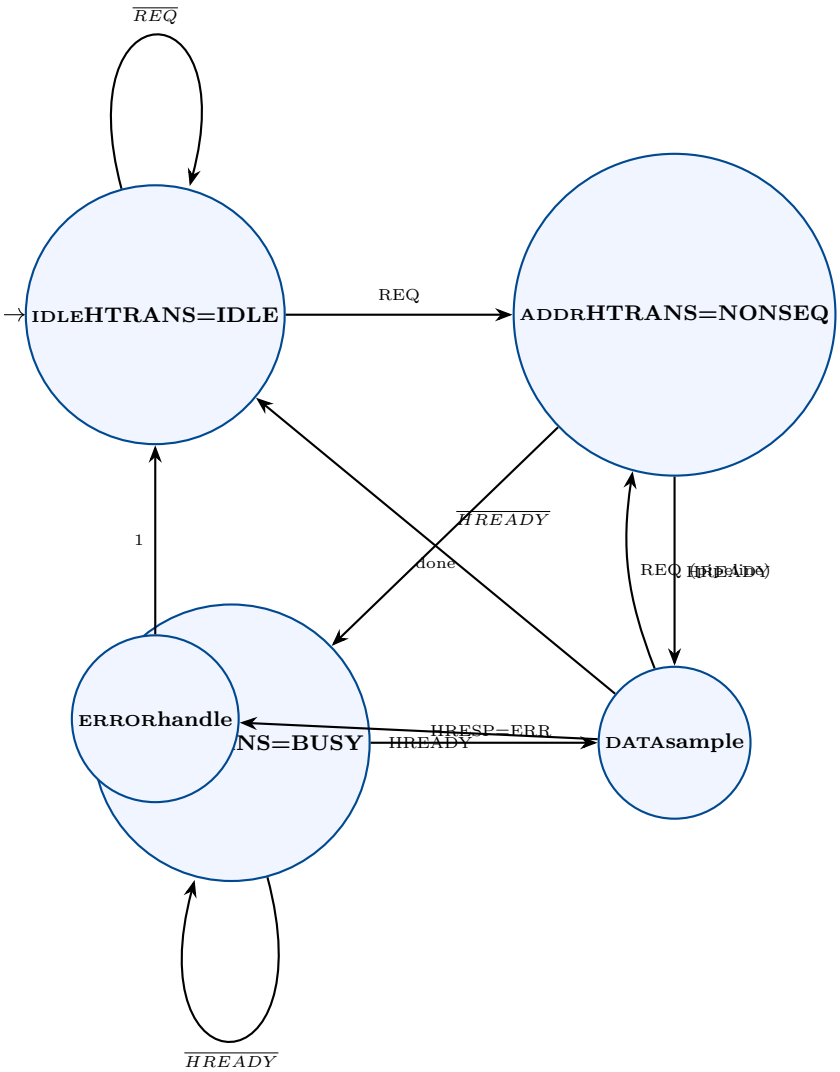


Figure 18: AHB-Lite Bus Master FSM

Key design considerations:

- Pipelined nature: address phase of next transfer overlaps data phase of current
- HREADY is critical — FSM must stall when slave is not ready
- Error handling must comply with AMBA spec (two-cycle error response)
- 5 states → 3 flip-flops, but one-hot encoding preferred in FPGAs

7 Complete Worked Example: Excitation Table to Circuit

Let us work through **Problem 6 (Traffic Light Controller)** completely.

7.1 Given State Diagram

Refer to Figure 8. States: $S_G = 00$, $S_Y = 01$, $S_R = 10$. Input: S (sensor). Unused: 11.

7.2 Complete Excitation Table

Table 7: Complete Traffic Light Excitation Table

Q_1	Q_0	S	Q_1^+	Q_0^+	D_1	D_0	G	Y	R
0	0	0	0	1	0	1	1	0	0
0	0	1	0	0	0	0	1	0	0
0	1	0	1	0	1	0	0	1	0
0	1	1	1	0	1	0	0	1	0
1	0	0	0	0	0	0	0	0	1
1	0	1	0	0	0	0	0	0	1
1	1	0	0	0	0	0	d	d	d
1	1	1	0	0	0	0	d	d	d

7.3 K-Map for D_1

	$S = 0$	$S = 1$
$Q_1Q_0 = 00$	0	0
$Q_1Q_0 = 01$	green!201	green!201
$Q_1Q_0 = 11$	gray!20d	gray!20d
$Q_1Q_0 = 10$	0	0

Group: $Q_0 \cdot \overline{Q_1}$ (with don't-cares, can simplify to Q_0)

$$\boxed{D_1 = Q_0}$$

7.4 K-Map for D_0

	$S = 0$	$S = 1$
$Q_1Q_0 = 00$	green!201	0
$Q_1Q_0 = 01$	0	0
$Q_1Q_0 = 11$	gray!20d	gray!20d
$Q_1Q_0 = 10$	0	0

$$\boxed{D_0 = \overline{Q_1} \cdot \overline{Q_0} \cdot \overline{S}}$$

7.5 Output K-Maps

Green ($G = \overline{Q_1} \cdot \overline{Q_0}$), **Yellow** ($Y = \overline{Q_1} \cdot Q_0$), **Red** ($R = Q_1 \cdot \overline{Q_0}$)

7.6 Final Circuit

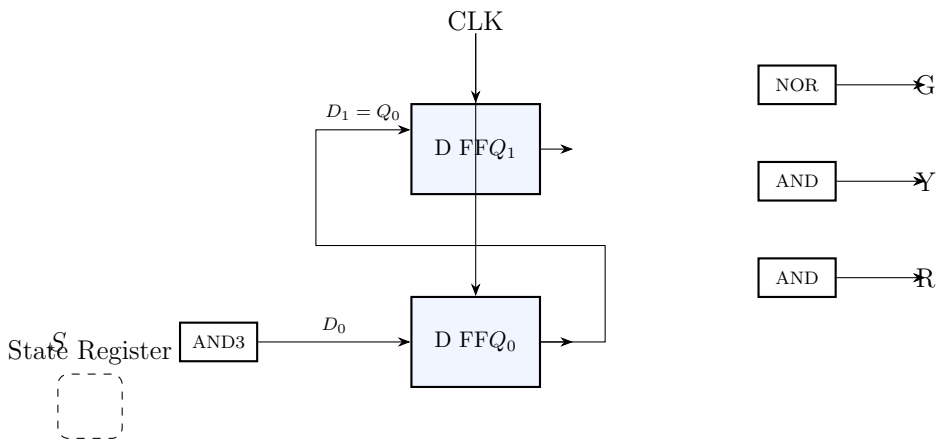


Figure 19: Complete Traffic Light Controller Circuit

8 Comparison: Moore vs. Mealy Implementation

Table 8: Moore vs. Mealy — Design Trade-offs

Aspect	**Moore**	**Mealy**
Output depends on	State only	State + Input
# States	Generally more	Generally fewer
Output timing	Changes with clock	Can change with input (async)
Glitch risk	Lower (synchronous)	Higher (input-dependent)
Output latency	One clock cycle more	Immediate
Design complexity	Easier	Slightly harder
Best for	Safety-critical, FPGA	Speed-critical, fewer states

9 Advanced Topics

9.1 One-Hot Encoding Implementation

In one-hot encoding, each state uses one flip-flop. For n states:

- n flip-flops (vs. $\lceil \log_2 n \rceil$ for binary)
- Next-state logic is very simple (often one gate per transition)
- Preferred for FPGA implementations (abundant flip-flops, limited routing)

Example: Traffic Light with one-hot (3 flip-flops for 3 states):
State $S_G = 001$, $S_Y = 010$, $S_R = 100$

$$D_G = Q_R \quad (\text{R} \rightarrow \text{G}) \tag{23}$$
$$D_Y = Q_G \cdot \overline{S} \quad (\text{G} \rightarrow \text{Y when no cars}) \tag{24}$$
$$D_R = Q_Y \quad (\text{Y} \rightarrow \text{R}) \tag{25}$$

Much simpler than binary encoding!

9.2 State Minimization Using Implication Chart

1. Create a triangular chart with all state pairs.
2. Mark pairs with different outputs as **not equivalent** ($\times$).
3. For remaining pairs, list implied equivalences needed.
4. Iterate: if an implied pair is marked $\times$, mark the current pair $\times$.
5. Repeat until no more changes.
6. Unmarked pairs are equivalent and can be merged.

9.3 Handling Unused States

For an FSM with 2^k possible states but only $n < 2^k$ used:

Safety-Critical Design

Always assign unused states a transition to the reset/idle state. In the excitation table, do NOT leave them as don't-cares for safety-critical systems. For non-critical applications, don't-cares can optimize logic.

9.4 Problem 16: Sequential Button Door Lock (Expert — Full Design)

9.4.1 Problem Statement

Sequential Button Door Lock

Design a Mealy FSM for a secure door lock.
Correct unlock sequence:

$$\boxed{S \rightarrow R \rightarrow B \rightarrow G \rightarrow R}$$

(Start, then Red, Blue, Green, Red)

Inputs (active-high, exactly one at a time):
S Start button
R Red button
B Blue button
G Green button

Outputs:

U **Unlock** — goes HIGH ($U=1$) only in state RED2 (correct full sequence entered). Otherwise $U=0$.

A **Activity** — goes HIGH ($A=1$) whenever *any* coloured button is pressed (R, B, or G), regardless of whether it is correct or wrong. $A=0$ when S is pressed or no button is pressed.

Rules:

1. Wrong coloured button at any step ⇒ return to WAIT immediately.
2. Pressing S mid-sequence ⇒ restart (go back to WAIT).
3. Repeated correct button (held down) ⇒ stay in current state (lenient).
4. No button pressed (idle cycle) ⇒ stay in current state.
5. After unlock, auto-return to WAIT on the next clock.

9.4.2 Understanding Output A (Activity Indicator)

What exactly is A?

A is a **purely combinational Mealy output**. It does *not* depend on which state you are in. It only depends on the *current input*:

$$\boxed{A = R + B + G}$$

Meaning: **any time a coloured button is pressed**, A lights up.

S	R	B	G	A	**Reason**
0	0	0	0	0	No button pressed (idle)
1	0	0	0	0	Start is not a colour
0	1	0	0	1	Red is a colour
0	0	1	0	1	Blue is a colour
0	0	0	1	1	Green is a colour

Use case: A could drive a feedback LED so the user knows "I pressed a coloured button." It gives no hint whether the colour was *correct* (security).

9.4.3 State Definitions

Table 9: States, Meanings, and Outputs

State	Meaning	U	A rule
WAIT	Idle; waiting for $S = 1$	0	$R + B + G$
START	Got S; expecting R	0	$R + B + G$
RED1	Got correct R; expecting B	0	$R + B + G$
BLUE	Got correct B; expecting G	0	$R + B + G$
GREEN	Got correct G; expecting R	0	$R + B + G$
RED2	Full sequence correct!	**1**	$R + B + G$
UNLOCK	Door open; next clk → WAIT	0	$R + B + G$

Note: $A = R + B + G$ in *every* state — it is state-independent.

9.4.4 State Diagram

The diagram is split into **three clear layers** so nothing overlaps.

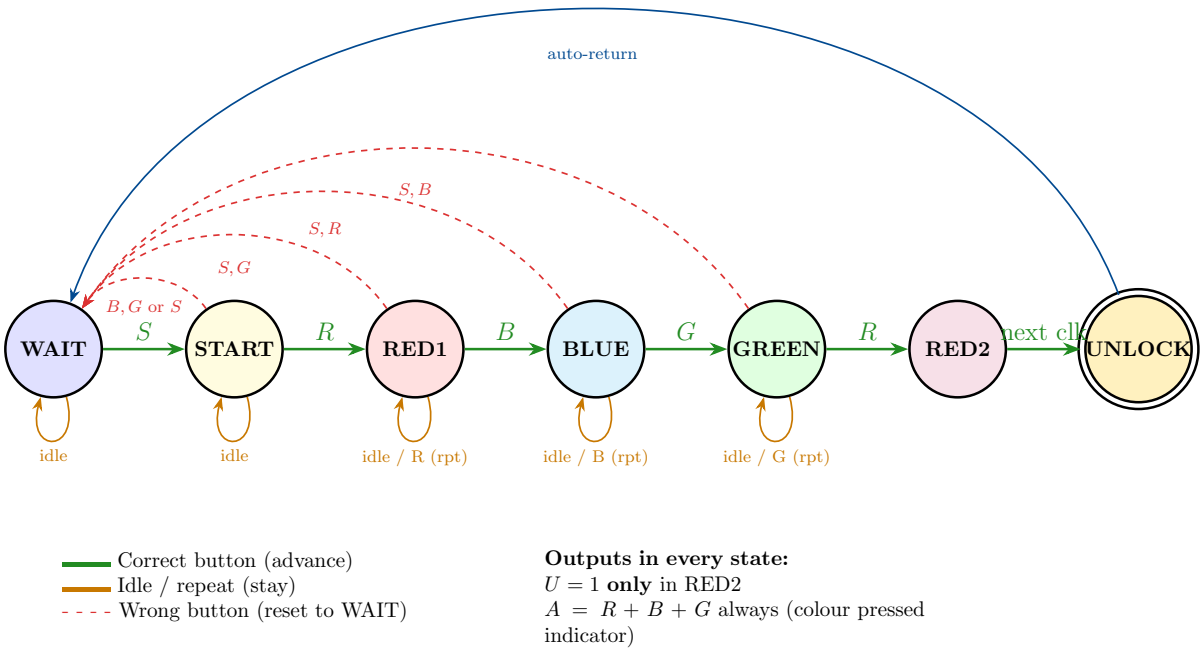


Figure 20: Sequential Button Lock — Complete State Diagram

9.4.5 Edge Cases — Every Scenario Explained

Table 10: All Edge-Case Scenarios

#	Scenario	State	U	A	Next State
1	Press R before S	WAIT	0	**1**	WAIT (ignored)
2	Press B before S	WAIT	0	**1**	WAIT (ignored)
3	Press G before S	WAIT	0	**1**	WAIT (ignored)
4	Press S (correct start)	WAIT	0	0	START
5	Idle (no button) in START	START	0	0	START (stay)
6	Press B in START (wrong)	START	0	**1**	WAIT (reset)
7	Press G in START (wrong)	START	0	**1**	WAIT (reset)
8	Press S again in START	START	0	0	WAIT (reset)
9	Press R in START (correct)	START	0	**1**	RED1
10	Press R again in RED1 (repeat)	RED1	0	**1**	RED1 (stay)
11	Press G in RED1 (wrong)	RED1	0	**1**	WAIT (reset)
12	Press B in RED1 (correct)	RED1	0	**1**	BLUE
13	Press R in BLUE (wrong)	BLUE	0	**1**	WAIT (reset)
14	Press B in BLUE (repeat)	BLUE	0	**1**	BLUE (stay)
15	Press G in BLUE (correct)	BLUE	0	**1**	GREEN
16	Press B in GREEN (wrong)	GREEN	0	**1**	WAIT (reset)
17	Press G in GREEN (repeat)	GREEN	0	**1**	GREEN (stay)
18	Press R in GREEN (correct)	GREEN	0	**1**	RED2
19	Arrive in RED2	RED2	**1**	*	UNLOCK
20	Leave UNLOCK (auto)	UNLOCK	0	0	WAIT
21	S pressed mid-sequence	any	0	0	WAIT (reset)
22	Idle cycle in any state	any	0	0	same (stay)
23	Button held 100 clocks	any	–	**1**	same (stay)
24	S-R-B-G-B (wrong at end)	GREEN	0	**1**	WAIT (B wrong)
25	Correct seq then immediate S	WAIT	0	0	START (new attempt)

* A depends on whether a colour button is being pressed at that moment.

Key Edge-Case Insights

1. A **fires even on wrong buttons.** If the user presses Red when Blue was expected, A still goes HIGH because a *colour* was pressed. A is not a "correct" indicator — it is a "colour-activity" indicator.
2. A **does NOT fire for Start.** S is not a colour, so $A = 0$ when only S is pressed.
3. A **does NOT fire on idle.** No button $\Rightarrow A = 0$.
4. **Repeated correct button = stay.** Holding Red in RED1 keeps the FSM in RED1 with $A = 1$, waiting for the next distinct press.
5. **Wrong colour = instant reset.** Even though $A = 1$ lights up, the FSM silently returns to WAIT.

6. U **= 1 for exactly one clock.** RED2 → UNLOCK → WAIT. The unlock pulse is brief.

9.4.6 State Assignment (Binary Encoding)

Table 11: State Encoding — 7 states, 3 flip-flops ($Q_2Q_1Q_0$)

State	Q_2	Q_1	Q_0	**Expects Next**
WAIT	0	0	0	S (Start)
START	0	0	1	R (Red)
RED1	0	1	0	B (Blue)
BLUE	0	1	1	G (Green)
GREEN	1	0	0	R (Red)
RED2	1	0	1	(done — unlock)
UNLOCK	1	1	0	(auto → WAIT)
(unused)	1	1	1	→ WAIT (safe)

9.4.7 Complete Excitation Table

Table 12: Full Excitation Table (D Flip-Flops)

Q_2	Q_1	Q_0	S	R	B	G	D_2	D_1	D_0	U	A	Transition
blue!8**WAIT (000)**												
0	0	0	1	0	0	0	0	0	1	0	0	→ START
0	0	0	0	1	0	0	0	0	0	0	**1**	stay (wrong, no S yet)
0	0	0	0	0	1	0	0	0	0	0	**1**	stay
0	0	0	0	0	0	1	0	0	0	0	**1**	stay
0	0	0	0	0	0	0	0	0	0	0	0	stay (idle)
yellow!8**START (001) — expects R**												
0	0	1	0	1	0	0	0	1	0	0	**1**	→ RED1 (correct!)
0	0	1	0	0	1	0	0	0	0	0	**1**	→ WAIT (B wrong)
0	0	1	0	0	0	1	0	0	0	0	**1**	→ WAIT (G wrong)
0	0	1	1	0	0	0	0	0	0	0	0	→ WAIT (S mid-seq)
0	0	1	0	0	0	0	0	0	1	0	0	stay (idle)
red!8**RED1 (010) — expects B**												
0	1	0	0	0	1	0	0	1	1	0	**1**	→ BLUE (correct!)
0	1	0	0	1	0	0	0	1	0	0	**1**	stay RED1 (R repeat)
0	1	0	0	0	0	1	0	0	0	0	**1**	→ WAIT (G wrong)
0	1	0	1	0	0	0	0	0	0	0	0	→ WAIT (S mid-seq)
0	1	0	0	0	0	0	0	1	0	0	0	stay (idle)
cyan!8**BLUE (011) — expects G**												
0	1	1	0	0	0	1	1	0	0	0	**1**	→ GREEN (correct!)
0	1	1	0	1	0	0	0	0	0	0	**1**	→ WAIT (R wrong)
0	1	1	0	0	1	0	0	1	1	0	**1**	stay BLUE (B repeat)
0	1	1	1	0	0	0	0	0	0	0	0	→ WAIT (S mid-seq)
0	1	1	0	0	0	0	0	1	1	0	0	stay (idle)
green!8**GREEN (100) — expects R**												
1	0	0	0	1	0	0	1	0	1	0	**1**	→ RED2 (correct!)
1	0	0	0	0	1	0	0	0	0	0	**1**	→ WAIT (B wrong)
1	0	0	0	0	0	1	1	0	0	0	**1**	stay GREEN (G repeat)
1	0	0	1	0	0	0	0	0	0	0	0	→ WAIT (S mid-seq)
1	0	0	0	0	0	0	1	0	0	0	0	stay (idle)
purple!8**RED2 (101) — UNLOCK**												
1	0	1	X	X	X	X	1	1	0	**1**	*	→ UNLOCK ($U=1$)
gold!10**UNLOCK (110)**												
1	1	0	X	X	X	X	0	0	0	0	*	→ WAIT (auto)
gray!10**(UNUSED 111)**												
1	1	1	X	X	X	X	0	0	0	0	0	→ WAIT (safe)

* $A = R + B + G$ always; shown as **1** where a colour is pressed.

9.4.8 Output Equations

Final Equations

Unlock output (U): Only HIGH in RED2 ($Q_2Q_1Q_0 = 101$):

$$\boxed{U = Q_2 \cdot \overline{Q_1} \cdot Q_0}$$

Activity indicator (A): Purely combinational (no state dependence):

$$\boxed{A = R + B + G}$$

This is extremely simple — just an OR gate on the three colour inputs. It is independent of the FSM state machine entirely.

9.4.9 Timing Diagram — Correct Sequence

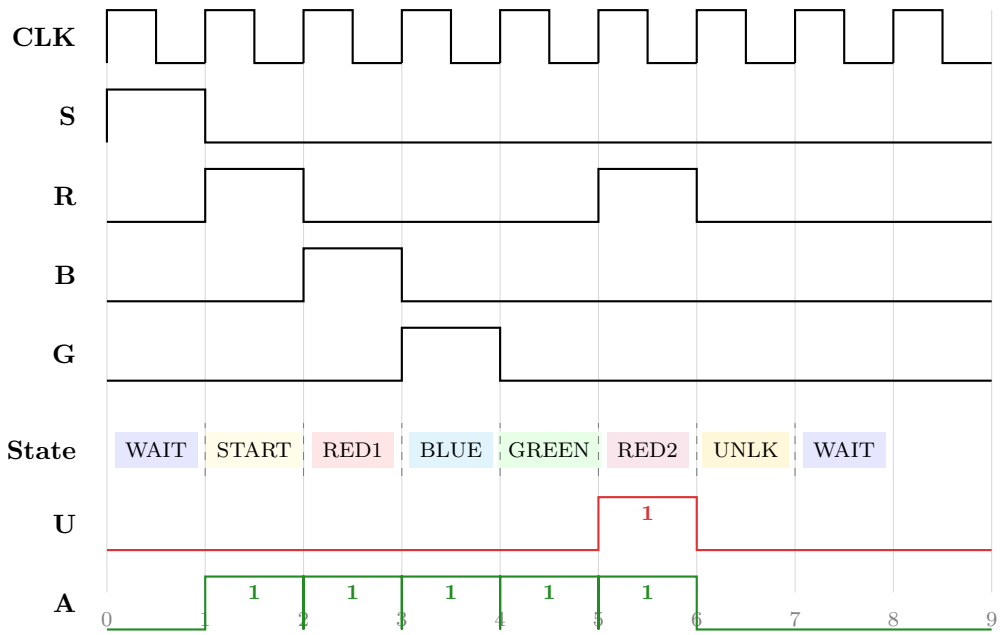


Figure 21: Timing Diagram: Correct unlock sequence $S \to R \to B \to G \to R$

9.4.10 Timing Diagram — Wrong Button Scenario

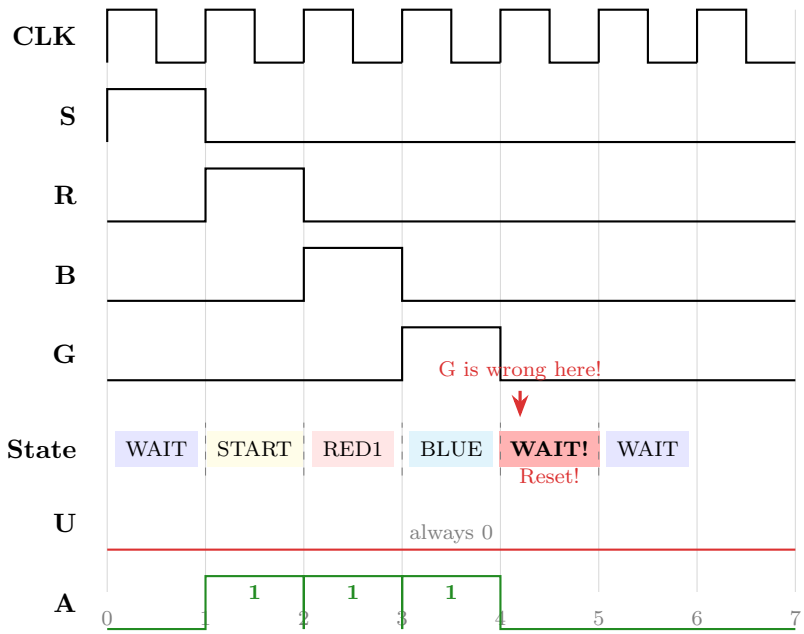


Figure 22: Timing Diagram: Wrong button (G pressed when B expected) — RESET, A still fires

9.4.11 Strict vs. Lenient Design Comparison

Table 13: Design Variations

Scenario	Strict (max security)	Lenient (our design)
Same button repeated	→ WAIT (error)	Stay in state
S pressed mid-seq	→ WAIT	→ WAIT
No timeout	—	Stay indefinitely
A on wrong colour	$A = 1$ (same)	$A = 1$ (same)
U duration	1 clock	1 clock

10 VHDL Template for FSM Design

This section provides a **universal VHDL template** you can reuse for *any* FSM. Just change the state names, inputs, outputs, and transition logic.

10.1 The Three-Process FSM Template

Three-Process Architecture (Recommended)

1. **Process 1 — State Register:** Sequential; updates state on clock edge.
2. **Process 2 — Next-State Logic:** Combinational; computes next state.
3. **Process 3 — Output Logic:** Combinational; computes outputs.

This cleanly separates concerns and is the standard taught in DSD courses.

10.2 Generic VHDL FSM Template

```
-----------------------------------------------------------------
-- GENERIC FSM TEMPLATE (Three-Process Style)
-- Usage: Copy this file, rename entity/architecture,
--        change TYPE state_type, and fill in the CASE blocks.
-----------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity fsm_template is
    Port (
        clk     : in  STD_LOGIC;
        reset   : in  STD_LOGIC;
        -- ===== ADD YOUR INPUTS HERE =====
        input_x : in  STD_LOGIC;
        -- ===== ADD YOUR OUTPUTS HERE =====
        output_y : out STD_LOGIC
    );
end fsm_template;

architecture Behavioral of fsm_template is

    -- ===== STEP 1: DEFINE YOUR STATES =====
    type state_type is (S0, S1, S2, S3);   -- Change these!
    signal current_state, next_state : state_type;

begin

    -------------------------------------------------------
    -- PROCESS 1: STATE REGISTER (Sequential)
    -- This is ALWAYS the same for every FSM.
    -- Just copy it. Never change it.
    -------------------------------------------------------
    STATE_REG: process(clk, reset)
    begin
        if reset = '1' then
            current_state <= S0;            -- Initial state
        elsif rising_edge(clk) then
```

```
        current_state <= next_state;
    end if;
end process STATE_REG;

-------------------------------------------------------
-- PROCESS 2: NEXT-STATE LOGIC (Combinational)
-- This is where YOUR design goes.
-- For each state, decide next_state based on inputs.
-------------------------------------------------------
NEXT_STATE_LOGIC: process(current_state, input_x)
    -- Include ALL inputs in sensitivity list!
begin
    -- Default: stay in current state (prevents latches!)
    next_state <= current_state;

    case current_state is

        when S0 =>
            if input_x = '1' then
                next_state <= S1;
            else
                next_state <= S0;   -- explicit stay
            end if;

        when S1 =>
            if input_x = '1' then
                next_state <= S2;
            else
                next_state <= S0;
            end if;

        when S2 =>
            next_state <= S3;       -- unconditional

        when S3 =>
            next_state <= S0;       -- return to start

        when others =>
            next_state <= S0;       -- catch unused states!

    end case;
end process NEXT_STATE_LOGIC;

-------------------------------------------------------
-- PROCESS 3: OUTPUT LOGIC (Combinational)
--
-- MOORE: output depends on current_state ONLY
-- MEALY: output depends on current_state AND inputs
-------------------------------------------------------

-- === MOORE VERSION ===
```

```
    MOORE_OUTPUT: process(current_state)
    begin
        -- Default outputs (prevents latches!)
        output_y <= '0';

        case current_state is
            when S0 => output_y <= '0';
            when S1 => output_y <= '0';
            when S2 => output_y <= '1';
            when S3 => output_y <= '1';
            when others => output_y <= '0';
        end case;
    end process MOORE_OUTPUT;

    -- === MEALY VERSION (use this OR Moore, not both) ===
    -- MEALY_OUTPUT: process(current_state, input_x)
    -- begin
    --     output_y <= '0';  -- default
    --     case current_state is
    --         when S0 =>
    --             if input_x = '1' then
    --                 output_y <= '1';
    --             end if;
    --         when others =>
    --             output_y <= '0';
    --     end case;
    -- end process MEALY_OUTPUT;

end Behavioral;
```

10.3 Template Rules — Never Break These

VHDL FSM Rules (Memorise These)

1. **Sensitivity list must be complete.**
   - State register: `(clk, reset)`
   - Next-state logic: `(current_state, all_inputs)`
   - Moore output: `(current_state)`
   - Mealy output: `(current_state, all_inputs)`
2. **Always assign default outputs** at the top of every combinational process. This prevents accidental **latches** (the #1 FSM bug).
3. **Always include** `when others` in every `case` statement to handle unused/illegal states.
4. **Never use** `rising_edge` in a combinational process. Only in the state register process.

5. **Never read outputs inside the architecture.** Use internal signals instead.
6. **Use enumerated types** (`type state_type is (...)`) — never raw `STD_LOGIC_VECTOR` for states. Let the synthesiser pick the encoding.

10.4 Applied Template: Button Lock in VHDL

```
-----------------------------------------------------------------
-- Problem 16: Sequential Button Door Lock
-----------------------------------------------------------------
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity button_lock is
    Port (
        clk   : in  STD_LOGIC;
        reset : in  STD_LOGIC;
        s     : in  STD_LOGIC;     -- start button
        r     : in  STD_LOGIC;     -- red button
        b     : in  STD_LOGIC;     -- blue button
        g     : in  STD_LOGIC;     -- green button
        u     : out STD_LOGIC;     -- unlock
        a     : out STD_LOGIC      -- activity (colour pressed)
    );
end button_lock;

architecture Behavioral of button_lock is

    type state_type is (WAIT_ST, START_ST, RED1_ST,
                        BLUE_ST, GREEN_ST, RED2_ST,
                        UNLOCK_ST);
    signal current_state, next_state : state_type;

begin

    -- Process 1: State Register
    STATE_REG: process(clk, reset)
    begin
        if reset = '1' then
            current_state <= WAIT_ST;
        elsif rising_edge(clk) then
            current_state <= next_state;
        end if;
    end process STATE_REG;

    -- Process 2: Next-State Logic
    NEXT_STATE_LOGIC: process(current_state, s, r, b, g)
    begin
        next_state <= current_state;   -- default: stay
```

```
    case current_state is

        when WAIT_ST =>
            if s = '1' then
                next_state <= START_ST;
            -- else stay (colour buttons or idle)
            end if;

        when START_ST =>
            if    r = '1' then next_state <= RED1_ST;   -- correct
            elsif b = '1' then next_state <= WAIT_ST;   -- wrong
            elsif g = '1' then next_state <= WAIT_ST;   -- wrong
            elsif s = '1' then next_state <= WAIT_ST;   -- restart
            -- else idle: stay
            end if;

        when RED1_ST =>
            if    b = '1' then next_state <= BLUE_ST;   -- correct
            elsif g = '1' then next_state <= WAIT_ST;   -- wrong
            elsif s = '1' then next_state <= WAIT_ST;   -- restart
            -- r = '1': stay (repeat); idle: stay
            end if;

        when BLUE_ST =>
            if    g = '1' then next_state <= GREEN_ST;  -- correct
            elsif r = '1' then next_state <= WAIT_ST;   -- wrong
            elsif s = '1' then next_state <= WAIT_ST;   -- restart
            -- b = '1': stay (repeat); idle: stay
            end if;

        when GREEN_ST =>
            if    r = '1' then next_state <= RED2_ST;   -- correct
            elsif b = '1' then next_state <= WAIT_ST;   -- wrong
            elsif s = '1' then next_state <= WAIT_ST;   -- restart
            -- g = '1': stay (repeat); idle: stay
            end if;

        when RED2_ST =>
            next_state <= UNLOCK_ST;  -- unconditional

        when UNLOCK_ST =>
            next_state <= WAIT_ST;    -- auto-return

        when others =>
            next_state <= WAIT_ST;    -- safety

    end case;
end process NEXT_STATE_LOGIC;

-- Process 3a: Unlock Output (Moore | depends on state only)
u <= '1' when current_state = RED2_ST else '0';
```

```
    -- Process 3b: Activity Output (Mealy | depends on inputs only)
    a <= r or b or g;

end Behavioral;
```

How to Reuse This Template for Any FSM

1. Copy the generic template.
2. Change `state_type` to list your states.
3. Change ports to your inputs/outputs.
4. Fill in the `case` block in Process 2 with your transitions.
5. Fill in Process 3 with your output logic.
6. **Done.** The state register (Process 1) never changes.

11 Quick Reference: FSM Design Checklist

Before You Submit Your Design

- ☐ All states have transitions for *all* input combinations
- ☐ Reset/initial state is clearly defined
- ☐ No unreachable or dead-end states
- ☐ Unused state combinations handled (safe state)
- ☐ State encoding is documented
- ☐ Excitation table matches state diagram exactly
- ☐ K-maps are correct (check minterms!)
- ☐ Boolean equations verified against truth table
- ☐ Output equations are correct (Moore: state only; Mealy: state + input)
- ☐ Circuit diagram matches equations
- ☐ Timing diagram verification attempted
- ☐ All flip-flops have clock and reset connections

12 Summary of All 16 Problems

Table 14: Overview of FSM Design Problems

#	Problem	Level	Type	States	FFs
1	Toggle Switch	Beginner	Moore	2	1
2	2-Bit Up Counter	Beginner	Moore	4	2
3	"11" Detector	Beginner	Mealy	2	1
4	Up/Down Counter	Beg–Int	Moore	8	3
5	Vending Machine	Intermediate	Moore	4	2
6	Traffic Light	Intermediate	Moore	3	2
7	Serial Adder	Intermediate	Mealy	2	1
8	"1010" Detector	Intermediate	Moore	5	3
9	Elevator Door	Int–Adv	Moore	4	2
10	SPI Master	Advanced	Moore	6	3
11	UART Receiver	Advanced	Moore	5	3
12	Washing Machine	Advanced	Moore	6	3
13	Combination Lock	Advanced	Mealy	3	2
14	Memory Controller	Advanced	Moore	5	3
15	AHB-Lite Bus	Expert	Moore	5	3
16	Sequential Button Lock	Expert	Mealy	7	3

— End of FSM Design Guide —